

Tema 4: Immersió i arbres binaris

4.1 La immersió

Quan una funció es crida a si mateixa de forma recursiva, la memòria demana un nou bloc (*frame*) per executar la seva instància particular. A l'exàmen no podem alterar la “signatura pública” (si et diuen fes `reverse(string s)`, no pots afegir arguments pel teu compte). L'estratègia de la **Immersió** respon d'aquesta manera: 1. Crear una segona funció auxiliar. 2. Fer que la funció pública pre-carregui aquesta funció d'immersió oculta.

Invertir String (`reverse`)

Necessitem un acumulador per guardar el *string* girat. Passant un simple segon paràmetre per immersió aconseguim portar el càlcul entre instàncies:

```
// 1. Funció immersa (auxiliar)
string reverse__(string s, string reversed) {
    if (s.empty()) return reversed;
    return reverse__(s.substr(1), s[0] + reversed);
}

// 2. Funció pública (Interfície original)
string reverse(string s) {
    return reverse__(s, "");
}
```

Fibonacci en $\mathcal{O}(n)$

La recursió exponencial convencional al calcular Fibonacci repetia crides sobre crides llastimosament provocant paràlisis matemàtiques de $\mathcal{O}(2^n)$. Amb una única Immersió baixem i solucionem el cost a linear $\mathcal{O}(n)$ passant pel viatge directament l'evolució dels dos últims nombres obtinguts.

```
int fibonacci__(int n, int a, int b) {
    if (n == 0) return a;
    return fibonacci__(n - 1, b, a + b);
}

int fibonacci(int n) {
    return fibonacci__(n, 0, 1);
}
```

4.2 L'arbre binari (`BinTree<T>`)

És una estructura de dades estrictament recursiva: o és un *buit absolut*, o té un node central (*arrel*) associat com a màxim a dos descendents exactes (*esquerra* i *dret*) que alhora, són considerats subarbres `BinTree` respectius en si mateixos.

[Gràfic interactiu disponible a la versió web de l'apunt]

Un arbre `BinTree` **NO es pot modificar**. Un cop fas el constructor i el tanques, mai podràs accedir a llicències com “*agafar la seva branca dreta nativa i esborrar-la amb un delete o set*”. Per alterar dades, s’opera **re-construint completament el mateix arbre com a Instància Nova** gràcies a aprofitar totes les parts antigues junt amb el canvi. (Mira sota, l’apartat 4.3).

4.3 Funcions bàsiques i de mutació

Anem a transformar els dos problemes més comuns a la classe `BinTree` (Saber l’altura total `height` o buscar si existeix una fulla `cerca`):

[Simulació interactiva disponible a la web]

Com l’arbre no té punters lliures o llicències d’assignació de variables natives com a Vectors positius, **qual-sevol operació que “modifiqui” un arbre** en teoria, a la pràctica C++, el que fa és reconstruir-lo sencer creant nous nodes per la zona o branca que hagi sofert el canvi! Revisions immutables.

4.4 Els recorreguts globals

Cerca en profunditat (DFS)

Baixar pel túnel fins el final abans d’escanejar lateralment.

- **Preordre:** *Arrel → Esquerre → Dret.*

[Simulació interactiva disponible a la web]

- **Inordre:** *Esquerre → Arrel → Dret.*

[Simulació interactiva disponible a la web]

- **Postordre:** *Esquerre → Dret → Arrel.*

[Simulació interactiva disponible a la web]

Cerca en amplada (BFS)

[Simulació interactiva disponible a la web]

4.5 Eficiència multitasca (`pair<A, B>`)

Si ens demanen resoldre dues coses a la vegada (ex: *Està sub-equilibrat? I quina altura té?*), llançar dues funcions de cerca separades provocarà un desastre d’eficiència a $\Theta(N^2)$. **La solució:** Buscar en una sola passada, retornant les dues dades alhora dins d’una tupla `std::pair` ($\Theta(N)$).

A continuació veiem com extreure tant la suma de tots els valors *com* la quantitat de nodes amb un sol `std::pair` per esbrinar la mitjana global:

[Simulació interactiva disponible a la web]

4.6 Llegir i Reconstruir Arbres

Als Jutges del laboratori rebràs els arbres representats en una línia de text plan (ex: 10 5 # # 14 # #), on un # indica “Sub-arbre Buit”. Aquí tens una utilitat ràpida auxiliar de conversió:

```
template <typename T>
T read_value(string text) {
    istringstream iss(text);
    T elem;
    iss >> elem;
    return elem;
}
```

1. Llegint en format Preordre (L’habitual)

Molt directe: L’arrel sempre és la primera a entrar pel cin. Després vénen els de l’esquerra, i finalment els de la dreta.

[Simulació interactiva disponible a la web]

2. Llegint en format Postordre (Amb Pila stack)

Si no hi ha més remei i t’ho donen en Postordre, la lectura normal falla perquè “la informació de l’arrel arriba a l’últim segon al teu teclat”. Haurem llegir-ho tot al revés apilant directament en un **stack** fins resoldre el camí sencer cap amunt: